DLDCA Lab 1: Verilog
Week 2

August 6, 2025

Introduction

In this lab, you will be learning about sequential circuits, what they are and how they work. This lab consists of 2 in-lab tasks, we have also included a presentation, in the `combinational_vs_sequential.pdf`, you should get started by going through it.

Necessary tools

We shall be using `icarus-verilog` alias `iverilog` inorder to compile verilog HDL. To visualise waveforms we shall use `gtkwave`.

Before starting the lab, ensure that both tools are installed in your system. This can be verified by running

```
$ iverilog
iverilog: no source files.
Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
           [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
$ gtkwave --version
GTKWave Analyzer v3.3.116 (w)1999-2023 BSI
```

```
GTKWAVE | Use the -h, --help command line flags to display help.
WM Destroy
```

Structure of submission/templates

Submission format(inlab):

```
your-roll-no/
|- inlab/
    |- task1/
        |- module.v (TODO)
        |- testbench.v
    |- task2/
        |- bugs.txt (TODO)
        |- buggy_modules.v (TODO)
        |- testbench.v
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Task 1: D-flip flop vs a buffer

So far, you may have come across the terms `wires` and `registers` in Verilog. At first glance, they may appear similar—both seem to serve the role of variables (like in C++) that hold values for later use. However, this is **not** the case. As we will understand through this task, they are fundamentally different objects with **distinct use cases**.

This misunderstanding arises from how people interpret Verilog. It is **not** a programming language in the conventional sense. Rather, it is a well-defined, text-based way to describe **digital circuits**.

Each `module` in Verilog represents a **hardware block**, and describes how it connects with other modules to form a larger circuit.

A `wire` in Verilog is analogous to a **physical wire**: it does not store any value but simply reflects the value driven by its source. Assigning a value to a wire is like

continuously driving a signal onto it—it immediately reflects changes as the source input changes.

On the other hand, a `reg` (short for "register") represents a **storage element**. It holds its value until explicitly updated. Even if the input changes, the value of the register remains unchanged until reassigned—typically on a **clock edge**, or when some specified condition is met.

This distinction is crucial:

- `wire` types are used for **combinational logic** and are driven using `assign` statements (continuous assignments).
- `reg` types are used in **sequential logic** to store state, and must be assigned values only inside `procedural blocks`, such as `always` blocks.

To demonstrate this, we will build:

- A **D Flip-Flop**, which updates its output only on a **clock edge** (rising or falling).
- A **buffer**, which simply replicates its input to the output without any storage.

Important: A `reg` **cannot** be assigned using `assign` statements—those are only valid for `wire` types. Continuous assignments are evaluated at all times, and outputs update immediately with changes in inputs. In contrast, values are assigned to `reg` variables only inside procedural blocks such as `always ... begin` blocks.

These procedural blocks execute their code **sequentially**, but only when a signal in their **sensitivity list** changes. For example:

```
always @(posedge clk) begin
    ... // this code is executed line by line every time
        // the value of 'clk' changes from 0 to 1
end
```

Refer to the given template code and fill it in order to replicate this logic. Make sure to test your implementation with the testbench as mentioned in the previous task.

Compilation instructions:

```
iverilog -o testbench testbench.v module.v
vvp testbench
```

Task 2: Shift register debugging

For this task, you have been provided with 4 implementations of a 4-bit shift register, each of which does not function as expected due to intentional bugs.

A 4-bit shift register in our implementation is a device which takes a 1-bit input, a clock input, and gives a 4-bit output. Every cycle, it **shifts** its register value by one bit to the left and inserts the input in the least significant bit (LSB).

An example of the expected behaviour of the shift register is shown below:

Clock Cycle	Input	Output
0	0	0000
1	1	0001
2	1	0011
3	0	0110
4	1	1101
5	0	1010

You are expected to use compile errors and gtkwave outputs to debug, find and fix these errors. These may include:

- Syntax errors (e.g., undeclared variables, wrong assignments)
- Incorrect shifting logic (e.g., wrong direction or wrong bit concatenation)
- Misuse of non-blocking vs blocking assignments
- Missing or incorrect sensitivity list

Apart from submitting a fixed implementation of the same, write all the bugs in each version of the register in a textfile called ‘bugs.txt‘.

Compilation instructions (Note compilation will fail due to syntax errors):

```
iverilog -o testbench testbench.v buggy_module.v
vvp testbench
```